

УДК 004.92, 004.4'2

Французов Александр Максимович
Frantsuzov Aleksandr Maksimovich

Магистрант
Master's student

Казанский национальный исследовательский институт им. А. Н. Туполева-КАИ
Kazan National Research Technical University named after A. N. Tupolev-KAI
Казань, Россия
Kazan, Russia

ОБЗОР И СРАВНИТЕЛЬНЫЙ АНАЛИЗ СОВРЕМЕННЫХ СРЕДСТВ НАГРУЗОЧНОГО ТЕСТИРОВАНИЯ ВЕБ-ПРИЛОЖЕНИЙ

REVIEW AND COMPARATIVE ANALYSIS OF MODERN WEB APPLICATION LOAD TESTING TOOLS

Аннотация: В статье представлен обзор и сравнительный анализ популярных инструментов для нагрузочного тестирования веб-приложений: Apache JMeter, Gatling, Locust и k6. Приведены основные характеристики каждого средства, сравнены их возможности по ключевым критериям. На основе анализа делаются выводы о применимости каждого инструмента в зависимости от требований проекта и квалификации команды.

Abstract: The article presents a review and comparative analysis of popular web application load testing tools: Apache JMeter, Gatling, Locust, and k6. The main features of each tool are described, and their capabilities are compared based on key criteria. Based on the analysis, conclusions are drawn regarding the applicability of each tool depending on the project requirements and the team's qualifications.

Ключевые слова: веб-приложение, нагрузочное тестирование, производительность, Apache JMeter, Gatling, Locust, k6.

Key words: web application, load testing, performance, Apache JMeter, Gatling, Locust, k6.

Введение

В современных условиях высокой конкуренции на рынке программного обеспечения тестировщикам все чаще необходимо

уделять внимание не только ключевым функциональным требованиям, но и нефункциональным, в числе которых – производительность и устойчивость системы под нагрузкой.

Если говорить о терминологии, то нагрузочное тестирование – это тестирование производительности, проводимый для оценки поведения компонента или системы при различных уровнях нагрузки, обычно в условиях низкой, типичной и пиковой загрузки [1].

Начинающие тестировщики, а нередко и продвинутые, сталкиваются с трудностями при выборе подходящего инструмента для проведения нагрузочного тестирования в конкретных условиях проекта.

Важно обозначить, что инструменты нагрузочного тестирования – это программные средства, используемые для имитации воздействия на систему одновременного множества пользователей, с целью оценки производительности в различных сценариях нагрузки. Они позволяют оценить время отклика системы, выявлять «узкие места» и анализировать поведение приложения в условиях, приближенных к реальным [2].

При выборе инструмента следует учитывать поддерживаемые протоколы, сложность создания сценариев, возможности мониторинга и анализа результатов, производительность, а также требования к масштабируемости тестирования [3].

Настоящая статья направлена на выявление преимуществ и недостатков четырех наиболее популярных на сегодняшний день инструментов нагрузочного тестирования - Apache JMeter, Gatling, Locust, k6.

Apache JMeter

Первым из рассматриваемых и одним из самых распространённых инструментов является Apache JMeter – приложение с открытым исходным кодом, созданное на Java и

предназначенное для тестирования функциональности и производительности под нагрузкой.

Согласно официальной документации JMeter, среди поддерживаемых протоколов значатся HTTP, HTTPS, SOAP/REST, FTP, JDBC, LDAP, JMS, SMTP/POP3/IMAP, TCP, выполнение нативных команд/скриптов и взаимодействие с Java-объектами [4]. Уже базовый набор впечатляет, но благодаря расширениям из каталога JMeter Plugins Manager его можно дополнительно расширить для тестирования самых разных систем.

Интерфейс JMeter основан на визуальной сборке тестов через компоненты, что делает его интуитивным и наглядным для начинающих. Тем не менее создание усложненных сценариев с глубокой параметризацией ограничено возможностями GUI, и в таких случаях JMeter предлагает возможность реализовать скрипты с помощью знания языков Groovy или BeanShell.

Для анализа результатов тестирования JMeter предлагает большое разнообразие встроенных средств мониторинга в реальном времени – таблицы, графики и логи запросов. Также предусмотрена возможность внешней интеграции с инструментами InfluxDB и Grafana.

Как и большинство Java-приложений, JMeter может испытывать проблемы с производительностью при больших профилях нагрузки, потребляя значительный объем оперативной памяти и ресурсом JVM. Для имитации десятков тысяч виртуальных пользователей потребуется либо производительная инфраструктура, либо тонкая настройка параметров JVM и грамотная организация распределённого режима (Master–Slave), что подчеркивает ограниченную масштабируемость инструмента.

Gatling

Gatling – это высокопроизводительное open-source решение для

нагрузочного тестирования, предназначенное для непрерывного тестирования нагрузки и интеграции в процесс разработки. Для работы с инструментом необходимо знание Scala, что может не подходить некоторым разработчикам без опыта работы с этим языком [5].

Gatling «из коробки» поддерживает классические HTTP/HTTPS, WebSocket и Server-Sent Events (SSE) [6]. Набор протоколов невелик, но благодаря открытому API можно подключать дополнительные протоколы или интегрироваться с существующими библиотеками на JVM, но без возможности расширения через графический каталог плагинов.

Невозможно реализовать с помощью графического интерфейса в Gatling также и создание сценариев, которые здесь описываются исключительно на Scala, что еще больше затрудняет работу новичков. Тем не менее эта первичная сложность из-за код-центричности облегчает версионирование тестов и реализация сложной логики без громоздких XML-файлов.

Для мониторинга Gatling делает ставку на единый, но информативный HTML-дашборд. По завершении теста автоматически генерируются интерактивные HTML-отчёты с графиками времени отклика, процентильных метрик и ошибок, без необходимости дополнительной настройки. Дополнительная визуализация доступна только через экспорт метрик в внешние системы, такие как InfluxDB и Graphite.

Главное преимущество Gatling – его производительность. Не просто так разработчики подчеркивают слово «высокопроизводительный»: движок Netty позволяет эффективно обслуживать тысячи виртуальных пользователей при низком потреблении CPU и памяти, что делает Gatling одним из лучших решений для крупных проектов.

Locust

Locust – лёгкий open-source инструмент для нагрузочного тестирования, написанный на языке Python.

Без дополнительных настроек поддерживаются исключительно HTTP и HTTPS [7]. Другие протоколы (WebSocket, MQTT и т. п.) подключаются вручную через библиотеки Python, что даёт гибкость, но требует дополнительной разработки.

Сценарии Locust представляют собой обычный Python-код, где каждая пользовательская задача оформляется как метод класса `HttpUser`. В этом есть как свои преимущества, как упрощение отладки и гибкость настройки сценариев, так и свои недостатки, как требование знания языка Python.

Минималистичный подход Locust сохраняется и в части мониторинга и отчётов: после запуска тестов пользователю доступен лишь веб-интерфейс с базовыми показателями, а дальнейший анализ доступен только после экспорта результатов в CSV. Интеграция с Prometheus + Grafana возможна, но требует разработки кастомных экспортёров.

Locust эмулирует тысячи одновременных пользователей с умеренным потреблением памяти и CPU, представляя собой сбалансированное решение по нагрузке среди других инструментов нагрузочного тестирования. Однако, для экстремальных нагрузок (десятки тысяч пользователей) может потребоваться распределённый запуск, который необходимо настроить с помощью встроенного режима Master–Worker. Распределение нагрузки по нескольким машинам настраивается без особых сложностей: каждому узлу достаточно Python-интерпретатора и доступа к коду сценариев.

K6

k6 (ранее известный как Grafana k6) – open-source инструмент нагрузочного тестирования, который делает значительный акцент на

облачные возможности, упрощающие нагрузочное тестирование для разработчиков и QA-инженеров [8].

Официально поддерживаемые протоколы – HTTP/HTTPS и gRPC [9]. WebSocket, GraphQL и SOAP доступны лишь частично, но у них нет полноценной поддержки, не покрывают сложные сценарии и не реализованы на уровне языка.

Работа со сценариями в k6 предельно схожа с Locust: тесты описываются в виде кода – каждая задача описывается функцией, а вспомогательные модули можно подключать с помощью импорта. Главное отличие между ними в языке – вместо Python используется JavaScript. К преимуществам подхода относятся широкая популярность JavaScript, простота версионирования и рефакторинг, но неприятно отсутствие GUI.

По умолчанию k6 выводит сводку результатов в консоль, однако существует экспорт в JSON и интеграция с InfluxDB и Grafana для основной визуализации.

k6 отличается низким потреблением памяти и быстрым стартом, что позволяет эмулировать тысячи виртуальных пользователей на одной машине. Для распределённого запуска используется либо облачный сервис k6 Cloud, либо контейнеризация через Docker – в обоих случаях масштабирование происходит легко благодаря простым CLI-командам и минимальным зависимостям.

Сравнение

Выведем ключевую информацию по инструментам нагрузочного тестирования в отдельную таблицу:

Таблица 1. Сравнение инструментов нагрузочного тестирования

Инструмент нагрузочного тестирования	Поддерживаемые протоколы	Описание сценариев	Мониторинг и отчёты	Производительность	Масштабирование
Apache JMeter	HTTP/HTTPS, SOAP/REST, FTP, JDBC, LDAP, JMS, SMTP/POP3/IMAP, TCP, нативные команды/скрипты, Java-объекты; расширяемый через плагины	GUI-редактор; скрипты на Groovy/Bean Shell	Встроенные Listeners (графики, таблицы, логи); интеграция с InfluxDB/Grafana; экспорт в CSV/XML	Высокое потребление памяти и CPU по JVM	Master-Slave по RMI; требует настройки и стабильной сети
Gatling	HTTP/HTTPS, WebSocket, SSE; расширяемый плагинами (gRPC и др.)	Скрипты на Scala-DSL	Автоматически генерируемый HTML-дашборд; экспорт в InfluxDB/Graphite	Эффективный Netty-движок, низкое потребление ресурсов	Распределённый запуск через контейнеры/скрипты; FrontLine для кластеризации
Locust	HTTP/HTTPS; прочие протоколы через сторонние Python-библиотеки	Python-скрипты (HttpUser с методами)	Веб-интерфейс с метриками; CSV-экспорт; интеграция с Prometheus/Grafana через кастомные экспортёры	Асинхронная модель на gevent, умеренное потребление ресурсов	Master-Worker без сложностей; каждому узлу нужен лишь Python
K6	HTTP/HTTPS, gRPC; частично WebSocket; расширяется через xk6-плагины	JavaScript – функции и модули через импорт	Консольная сводка; экспорт в JSON; интеграция с InfluxDB/Grafana	Go-рантайм, очень низкое потребление памяти и CPU	k6 Cloud; Docker-контейнеры; простые CLI-команды

Заключение

Комплексный подход к выбору инструмента позволяет более

точно адаптировать нагрузочное тестирование к конкретным требованиям проекта и обеспечить надежность функционирования системы в различных условиях [10].

Apache JMeter подойдёт новичкам благодаря наглядному GUI и широкой поддержке протоколов, но при высоких нагрузках требует мощной инфраструктуры и настройки.

Gatling будет отличным выбором для опытных команд, особенно если важны высокая производительность и интеграция в CI/CD, однако требует знания Scala и не имеет GUI.

Locust подойдёт тем, кто работает с Python и хочет гибко описывать сценарии в коде, особенно в условиях умеренной нагрузки. Для экстремальной нагрузки потребуется распределённый запуск.

k6 выделяется простотой масштабирования, быстрой интеграцией с облаком и низкими системными требованиями, а также удобством для тех, кто знаком с JavaScript.

Библиографический список:

1. Load Testing // ISTQB Glossary. URL: https://glossary.istqb.org/en_US/term/load-testing?term=load%20testing&exact_matches_first=true (дата обращения: 09.05.2025).
2. П.А. Редкин, А.С. Алёшкин Программный комплекс распределенного тестирования веб-приложений // International Journal of Open Information Technologies. – 2024. – №. vol. 12, no. 4. – С. 125-132 (дата обращения: 10.05.2025).
3. Бевзенко Сергей Александрович Процесс автоматизации нагрузочного тестирования веб-систем: основные аспекты // Universum: технические науки. – 2023. – №. №8-1 (113). – С. 20-22 (дата обращения: 10.05.2025).
4. What can I do with it? // Apache JMeter. URL:

<https://jmeter.apache.org/> (дата обращения: 10.05.2025).

5. Zarubaika A Implementation of fintech apis - basics how to setup projects // Инновации и инвестиции. – 2023. – №. №7. – С. 222-225.

6. Protocols // Gatling. URL: <https://docs.gatling.io/reference/script/protocols/http/protocol/> (дата обращения: 11.05.2025).

7. Testing other systems/protocols // Locust. URL: https://docs.locust.io/en/stable/testing-other-systems.html?utm_source=chatgpt.com (дата обращения: 11.05.2025).

8. Инструменты для распределенного нагрузочного тестирования: сравнение, особенности использования // ПерформансЛаб. URL: <https://www.performance-lab.ru/blog/instrumenty-dlya-raspredelennogo-nagruzochnogo-testirovaniya> (дата обращения: 11.05.2025).

9. Protocols // GrafanaLabs. URL: <https://grafana.com/docs/k6/latest/using-k6/protocols/> (дата обращения: 11.05.2025).

10. Бевзенко С.А. Основные виды классификации нагрузочного тестирования // Вестник науки. – 2024. – №. vol. 4, no. 1 (70). – С. 427-432 (дата обращения: 10.05.2025).